

IMN269 - Vision tridimensionnelle

Chapitre 7 Reconstruction 3D

Olivier Godin

Université de Sherbrooke

30 juillet 2025

- 1 Maillage et triangulation
- 2 Enveloppe convexe 3D

1 Maillage et triangulation

- Généralités
- Diagrammes de Voronoï
- Triangulation de Delaunay

2 Enveloppe convexe 3D

1 Maillage et triangulation

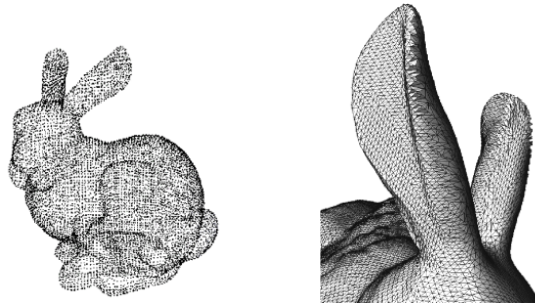
- Généralités
- Diagrammes de Voronoï
- Triangulation de Delaunay

2 Enveloppe convexe 3D

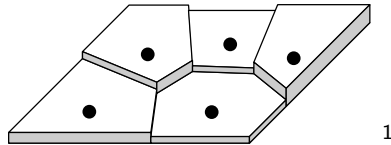
À partir d'un nuage de points 3D, on veut **interpoler la surface qui sous-tend le nuage**. Cela revient à trouver le modèle qui colle le mieux aux données.



On souhaite interpoler une surface 2D $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ où la valeur $f(x,y)$ est connue **seulement** en **un nombre fini de points**, que l'on notera P , un sous ensemble du domaine $A \subseteq \mathbb{R}^2$ de f .

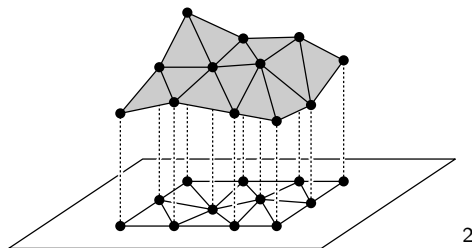


Une approche naïve consiste à assigner à chaque coordonnées de A la valeur de f au point $p \in P$ le plus près.



Une meilleure approche consiste à déterminer une **triangulation de P** . Dans une triangulation, chaque point $p_i = (x_i, y_i) \in P$ est élevé à sa hauteur $f(x_i, y_i)$ et ceux-ci sont **reliés par des triangles en 3D**.

Une triangulation est une **subdivision planeaire de A** où chaque division est un triangle dont les sommets sont les éléments de P .



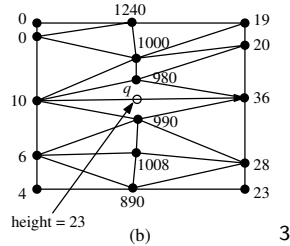
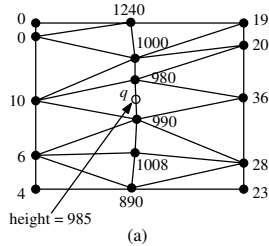
De plus, dans une triangulation T de P , il est **impossible d'ajouter un segment connectant deux points** sans intersecter un segment existant.

Plusieurs questions restent sans réponse :

- Est-ce qu'une triangulation pour un ensemble de points P **existe toujours** ?
- Cette triangulation est-elle **unique** ?
- Si elle ne l'est pas, comment savoir si une triangulation est **meilleure qu'une autre** ?

Comme **on ne dispose des valeurs de $f(x,y)$ qu'à certaines positions**, toutes les triangulations de P pourraient être perçues comme étant **aussi valides les unes que les autres**... Ce n'est toutefois pas le cas.

À titre d'exemple, considérons **deux triangulations d'un même ensemble de points**, où les valeurs numériques associées aux sommets sont les valeurs de $f(x,y)$ à ces points.



3

Les deux triangulations sont **identiques à l'exception d'une arête**, mais laquelle serait *la meilleure* ?

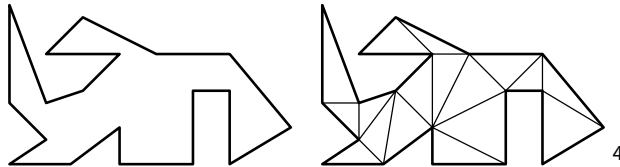
Comment **convertir cette intuition en un critère strict** qui nous dirait que (a) est une meilleure triangulation que (b) ?

Le problème de la triangulation (b) est que la valeur interpolée au point q est donnée par deux sommets assez éloignés de q . Une telle situation engendrera des **petits angles dans la triangulation** et c'est cela qu'on tentera d'éviter.

Partant du principe que les petits angles doivent être évités dans une triangulation, on cherchera à **trier celles-ci selon leur plus petit angle**.

Comme il existe un nombre fini de triangulations pour un ensemble de points P donné, on pourra trouver **celle qui maximise l'angle minimal**. Ce sera la triangulation recherchée.

La séparation d'un polygone en triangles à l'aide d'un **nombre maximal de diagonales sans intersection** est une triangulation du polygone.



Les triangulations ne sont, en général, **pas uniques**.

Le théorème suivant se veut rassurant :

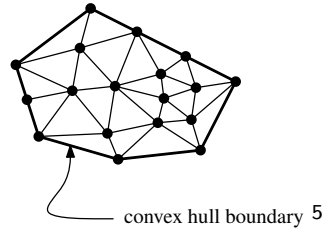
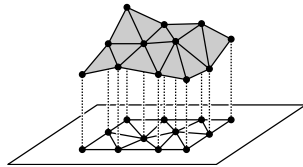
Théorème 3.1

Tout polygone simple admet une triangulation, et toute triangulation d'un polygone simple à n côtés comporte exactement $n - 2$ triangles.

En effet,

- il garantit l'existence d'une triangulation ; et
- il détermine la complexité de celle-ci.

Dans le cas d'un nuage de points 3D, on **projette ceux-ci sur un plan** et l'enveloppe convexe 2D des points projetés est le polygone A dont on cherche la triangulation.



Soit $\mathcal{T}$ une triangulation d'un ensemble de points P composée de m triangles. On s'intéresse aux **$3m$ angles formant les m triangles de $\mathcal{T}$** triés en ordre croissant.

On note par

$$\alpha_1, \alpha_2, \dots, \alpha_{3m}$$

cette séquence d'angles, avec $\alpha_i \leq \alpha_j$ si $i < j$.

On appelle $\alpha(\mathcal{T}) = (\alpha_1, \alpha_2, \dots, \alpha_{3m})$ le **vecteur-angle** de $\mathcal{T}$.

Soit $\mathcal{T}'$ une autre triangulation de P , et soit $\alpha(\mathcal{T}') = (\alpha'_1, \alpha'_2, \dots, \alpha'_{3m})$ son vecteur-angle. On dira que $\alpha(\mathcal{T}) > \alpha(\mathcal{T}')$ s'il existe une position i , avec $1 \leq i \leq 3m$, telle que

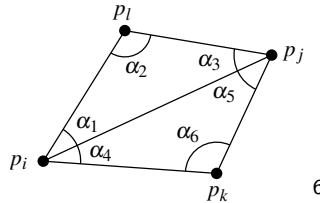
$$\alpha_j = \alpha'_j \quad \text{pour tout } j < i \quad \text{et} \quad \alpha_i > \alpha'_i.$$

Une triangulation de P est dite **angle-optimale** si $\alpha(\mathcal{T}) > \alpha(\mathcal{T}')$ pour toutes les autres triangulations $\mathcal{T}'$ de P .

Une triangulation angle-optimale **maximisera le plus petit angle** parmi l'ensemble des triangulations possibles.

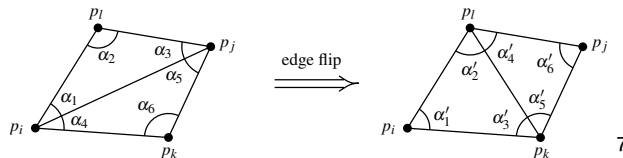
Considérons une arête $e = \overline{p_i p_j}$ faisant partie d'une triangulation $\mathcal{T}$ d'un ensemble de points P .

Si e ne fait pas partie de l'enveloppe convexe de la triangulation, on sait qu'elle est partagée par deux triangles $\triangle p_i p_j p_k$ et $\triangle p_i p_j p_l$.



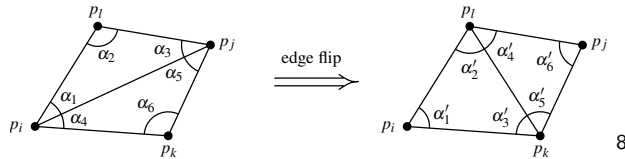
6

Une nouvelle triangulation $\mathcal{T}'$ de P peut alors être obtenue en **remplaçant l'arête $\overline{p_i p_j}$ par l'arête $\overline{p_k p_l}$** .



Ainsi, les seules différences entre les deux vecteur-angles sont les six angles $\alpha_1, \dots, \alpha_6$ dans $\alpha(\mathcal{T})$ qui sont remplacés par $\alpha'_1, \dots, \alpha'_6$ dans $\alpha(\mathcal{T}')$.

On dira que l'arête $e = \overline{p_i p_j}$ est **illégal** si $\min_{1 \leq i \leq 6} \alpha_i < \min_{1 \leq i \leq 6} \alpha'_i$.



En d'autres mots, si une arête est illégale, on peut augmenter localement la valeur du plus petit angle en permutant les arêtes.

Soit $\mathcal{T}$ une triangulation possédant une arête illégale e et soit $\mathcal{T}'$ la triangulation obtenue en permutant e . On aura alors que $\alpha(\mathcal{T}') > \alpha(\mathcal{T})$.

On définit alors le concept de **triangulation légale** comme étant une triangulation **ne contenant aucune arête illégale**.

De plus, toute triangulation angle-optimale est légale.

En utilisant cette définition, **l'obtention d'une triangulation légale à partir d'une triangulation initiale** est fort simple : il suffit de faire permuter des arêtes jusqu'à ce qu'elles soient toutes légales.

Comme il existe un nombre fini de triangulations pour un ensemble de points P , cette méthode fournira assurément une triangulation légale.

Cela dit, cette technique est trop lente pour être intéressante en pratique. De plus, **on ne peut garantir l'obtention d'une triangulation angle-optimale** par cette méthode, puisque le résultat obtenu dépend du point de départ.

On prendra plutôt un long détour pour trouver la triangulation souhaitée...

1 Maillage et triangulation

- Généralités
- Diagrammes de Voronoï
- Triangulation de Delaunay

2 Enveloppe convexe 3D

Soit P un ensemble de n points distincts sur le plan, appelés sites.

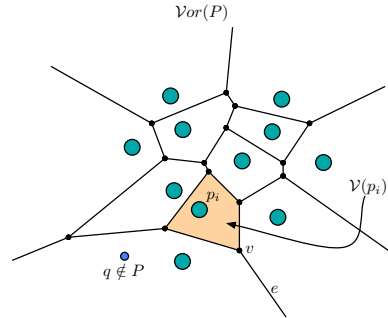
Le **diagramme de Voronoï** de P est la **subdivision du plan** en n cellules (une pour chaque site) telle qu'un point $q \notin P$ est dans la cellule correspondante à un site $p_i \in P$ si et seulement si

$$d(q, p_i) < d(q, p_j) \quad \text{pour tout } p_j \in P \quad \text{avec} \quad i \neq j,$$

où $d(p, q)$ est la distance euclidienne entre les points p et q .

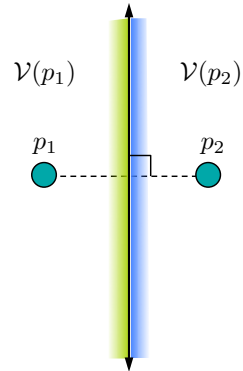
Dans le diagramme de droite,

- $\mathcal{V}or(P)$ est le diagramme de Voronoï associé à P
- $\mathcal{V}(p_i)$ est la cellule associée au site p_i
- q est un point quelconque absent de P
- e est une arête de Voronoï
- v est un sommet de Voronoï

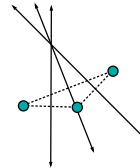


Si P contient seulement deux points, p_1 et p_2 , alors la droite bissectrice sépare le plan en 2 demi-plans, $h(p_1, p_2)$ et $h(p_2, p_1)$.

De plus, **cette droite est la seule arrête du diagramme de Voronoï associé à P .**

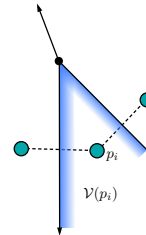


Lorsqu'on ajoute un point à P , on ajoute **une bissectrice pour chacune des paires de points**.

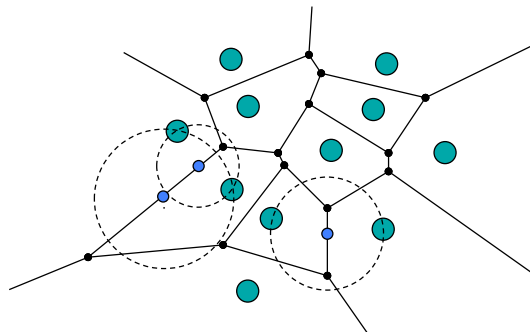


Une cellule est l'**intersection de plusieurs demi-plans** :

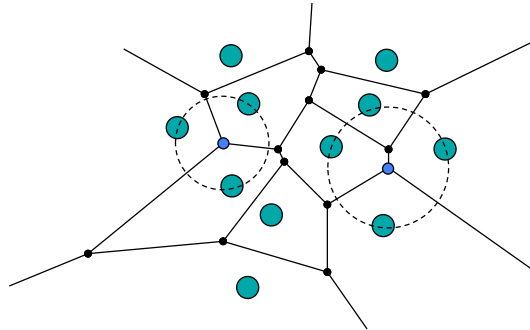
$$\mathcal{V}(p_i) = \bigcap_{\substack{1 \leq j \leq n \\ i \neq j}} h(p_i, p_j)$$



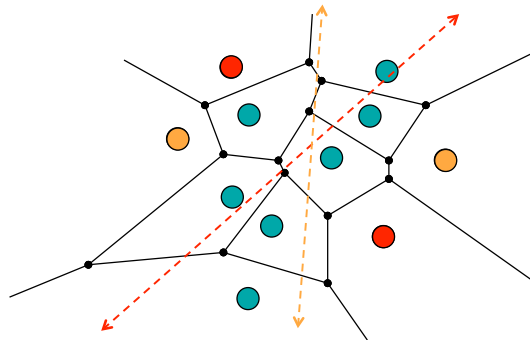
Un point $q \notin P$ est **sur une arête** de $\mathcal{Vor}(P)$ entre les sites p_i et p_j si et seulement si le plus grand cercle vide centré en q **touche seulement à p_i et p_j** .



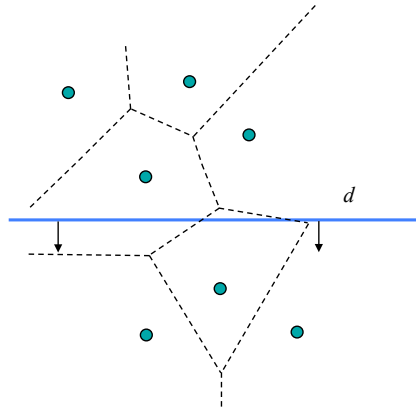
Un point $q \notin P$ est **un sommet** de $\mathcal{Vor}(P)$ entre les sites p_i et p_j si et seulement si le plus grand cercle vide centré en q **touche seulement à au moins trois sites**.



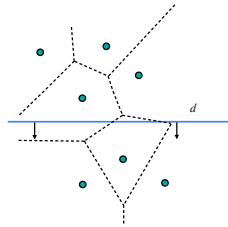
Notons que ce ne sont **pas** toutes les bissectrices ni toutes les intersections de bissectrices qui sont dans $\mathcal{Vor}(P)$.



L'**algorithme de Fortune** est une approche par balayage d'une droite d de haut en bas de l'ensemble des sites.

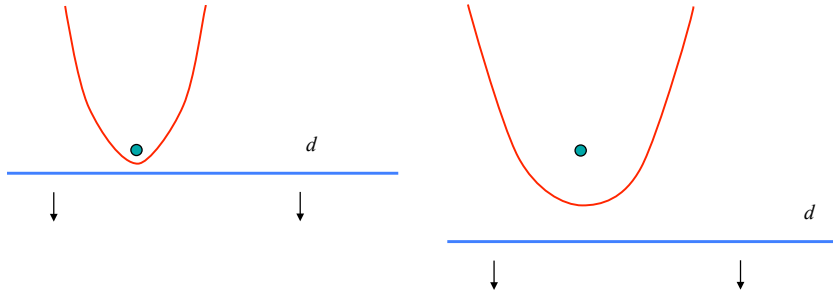


L'idée est de maintenir une représentation des positions des points qui sont **plus près d'un site p_i au dessus de d que de d .**

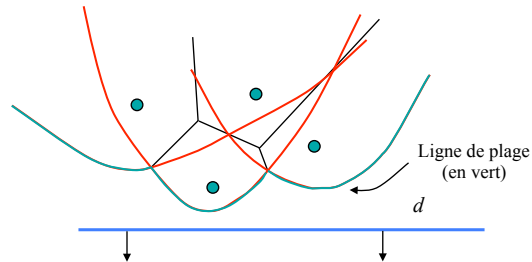


Cette représentation ne changera plus, car un tel point est **forcément plus près de d que de tous les sites en dessous de d .**

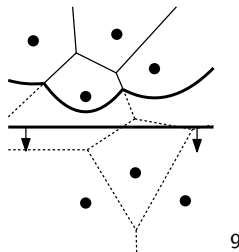
On peut caractériser analytiquement la ligne d'équidistance entre un point et une droite par **une** parabole qui s'évase à mesure que la droite s'éloigne du point.



Si on a plusieurs sites, chacun d'eux est représenté par une parabole différente. La ligne de front des paraboles est appelée **ligne de plage**.



Une arête du diagramme de Voronoï correspond aux intersections des paraboles. La position des intersections évolue à mesure que d descend.



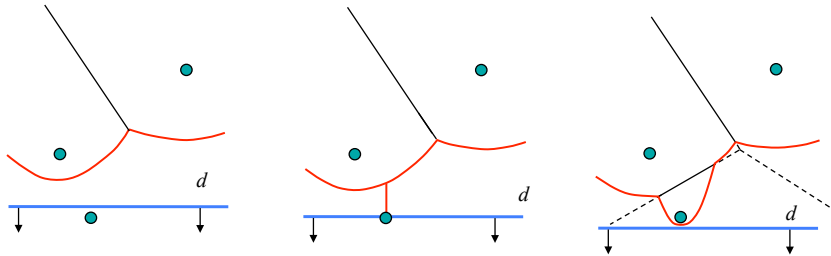
9

Ce qui est au dessus de la ligne de plage ne change plus.

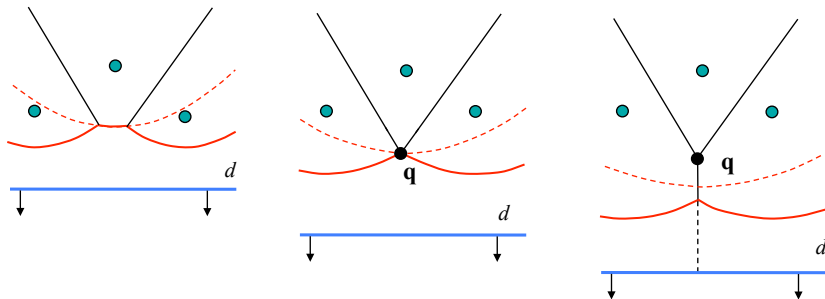
À mesure que la droite d descend et parcourt l'ensemble des sites, **deux événements peuvent survenir** :

- d rencontre un site
- un arc de parabole disparaît

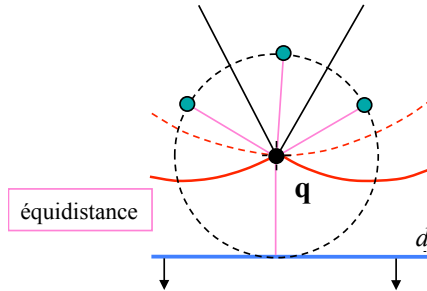
Lorsque d rencontre un site, **une nouvelle parabole est créée**, ce qui modifie la ligne de plage, et **une nouvelle arête** s'ajoute au diagramme.



Lorsqu'un arc de parabole diminue et disparaît au point q , **un nouveau sommet et une nouvelle arête** s'ajoutent au diagramme.



De plus, lors de la disparition d'un arc de parabole au point q , **au moins trois sites sont sur la frontière d'un cercle vide** centré en q .



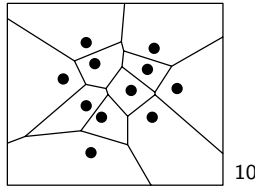
Pour l'implantation de l'algorithme de Fortune pour la construction d'un diagramme de Voronoï, on utilise deux structures de données :

- Un **arbre binaire de recherche** balancé pour stocker la ligne de plage ;
- Une **liste d'arêtes doublement chaînée** pour stocker le diagramme de Voronoï en construction.

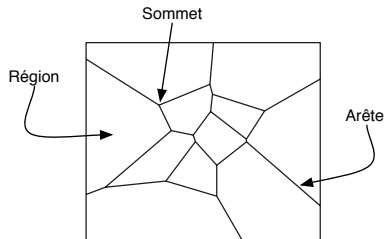
La **liste d'arêtes doublement chaînée** (*doubly-connected edge list*, ou DCEL), à la base, est une structure élaborée pour stocker une **subdivision du plan**.

Ainsi, une DCEL sert à représenter en mémoire **une subdivision plane**, mais son fonctionnement se prête très bien au stockage d'un diagramme de Voronoï.

Pour y arriver, on définit un rectangle englobant tous les points formant le diagramme de Voronoï :



Une DCEL contient un enregistrement pour chaque **région**, chaque **arête** et chaque **sommet** du diagramme de Voronoï.

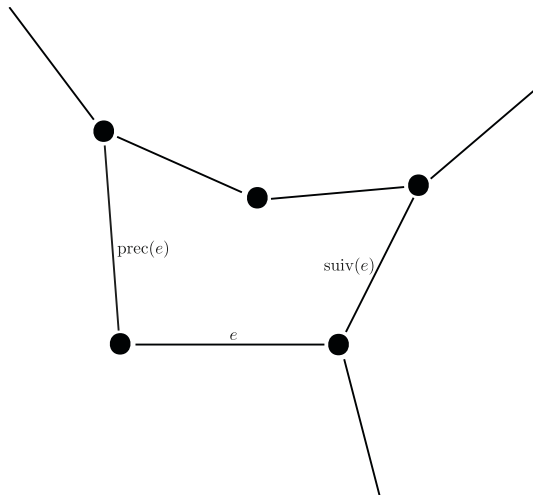


L'information géométrique et topologique contenue dans la DCEL doit permettre d'effectuer les opérations de base suivantes :

- **Faire le tour d'une région** en passant par chacune de ses arêtes ;
- **Accéder à une région à partir d'une région adjacente** en connaissant l'arête commune ;
- **Visiter toutes les arêtes** ayant comme extrémité un point donné.

Pour pouvoir effectuer la première tâche, c'est-à-dire **se promener d'une arête à l'autre** autour d'une région, on doit stocker **un pointeur de l'arête vers la suivante**.

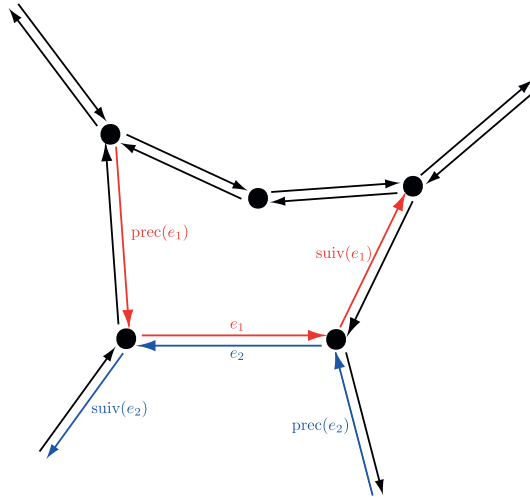
Qui plus est, ce n'est pas très forçant de permettre en plus le parcours dans l'autre sens. Il suffit alors de conserver aussi **un pointeur vers l'arête précédente**.



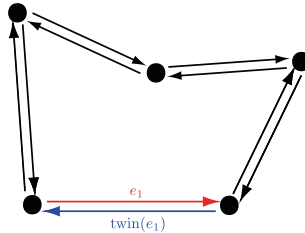
Comme une arête se trouve habituellement sur la frontière de deux régions, on devra stocker **deux paires de pointeurs**.

Il devient alors pertinent de voir les deux côtés d'une arête $\vec{e}$ **comme deux demi-arêtes distinctes** $\vec{e}_1$ et $\vec{e}_2$, de manière à définir sans ambiguïtés les deux paires d'arêtes précédente et suivante, dans le sens antihoraire (par convention).

Cela a aussi comme conséquence plaisante qu'une **demi-arête sera sur la bordure d'une seule région**.



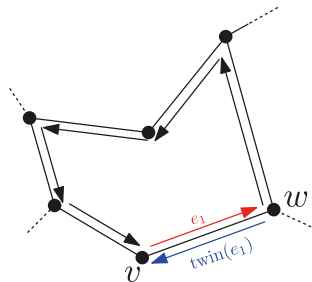
Les deux demi-arêtes associées à une arête sont appelés **jumeaux**.



Soit une demi-arête $\vec{e}$. Définir la demi arête suivante dans le sens antihoraire a pour conséquence que la région bornée par celle-ci se trouvera toujours du côté gauche.

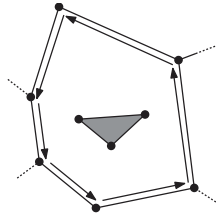
Comme les arêtes sont orientées, on peut parler de **point d'origine** et de **point d'arrivée** de celles-ci.

Si une arête $\vec{e}$ possède v comme point d'origine et w comme point d'arrivée, alors son **arête jumelle** a w comme point de départ et v comme point d'arrivée.



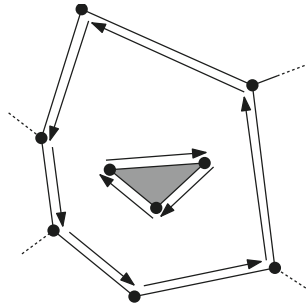
Finalement, un enregistrement de région dans la liste doit posséder **un pointeur vers une demi-arête sur sa frontière**, de manière à permettre d'en faire le tour.

Une situation qui n'a pas encore été gérée est celle des **trous dans une région**.



Dans ce cas, en parcourant les arêtes formant le trou dans le sens antihoraire, on aura que **la région se trouvera à la droite** de celles-ci.

Comme il est préférable de toujours avoir les régions du même côté des arêtes (à gauche), on choisira de **parcourir les arêtes dans le sens horaire** dans le cas d'un trou dans une région.



La gestion des trous dans les régions signifie aussi que le fait de **posséder un unique pointeur vers une des demi-arêtes** composant son contour n'est plus suffisant.

Il est plutôt nécessaire pour une région de posséder **un pointeur vers une demi-arête de chaque frontière qui la compose**, que celle-ci soit associée au contour ou à un trou dans la région.

En résumé, une DCEL compte **trois collections d'enregistrements** : une pour les sommets, une pour les régions et une pour les demi-arêtes.

L'enregistrement pour un sommet v comprend les coordonnées de v ainsi qu'un pointeur vers une quelconque demi-arête ayant v comme extrémité.

L'enregistrement pour une région f stocke quant à elle un pointeur vers une quelconque demi-arête sur sa bordure, de même qu'une liste de pointeurs vers une demi arête quelconque faisant partie de la frontière de chaque trou dans la région.

Finalement, l'enregistrement pour une demi-arête $\vec{e}$ doit inclure

- Un pointeur vers son sommet d'origine ;
- Un pointeur vers sa demi-arête jumelle ;
- Un pointeur vers la région qu'elle borne.

Il n'est toutefois pas nécessaire de stocker **un pointeur vers son sommet d'arrivée**, puisque celui-ci correspond à l'origine de la demi-arête jumelle.

L'enregistrement d'une demi-arête doit aussi contenir **des pointeurs vers les arêtes suivante et précédente**.

Les enregistrements pour les sommet et les arêtes occupent **une quantité de mémoire fixe**, tandis que l'espace occupé par un enregistrement de région est **variable** en raison de la liste de pointeurs vers les trous dans la région.

Algorithm VORONOIDIAGRAM(P)

Input. A set $P := \{p_1, \dots, p_n\}$ of point sites in the plane.

Output. The Voronoi diagram $\text{Vor}(P)$ given inside a bounding box in a doubly-connected edge list $\mathcal{D}$.

1. Initialize the event queue $\mathcal{Q}$ with all site events, initialize an empty status structure $\mathcal{T}$ and an empty doubly-connected edge list $\mathcal{D}$.
2. **while** $\mathcal{Q}$ is not empty
3. **do** Remove the event with largest y -coordinate from $\mathcal{Q}$.
4. **if** the event is a site event, occurring at site p_i
5. **then** HANDLESITEEVENT(p_i)
6. **else** HANDLECIRCLEEVENT(γ), where γ is the leaf of $\mathcal{T}$ representing the arc that will disappear
7. The internal nodes still present in $\mathcal{T}$ correspond to the half-infinite edges of the Voronoi diagram. Compute a bounding box that contains all vertices of the Voronoi diagram in its interior, and attach the half-infinite edges to the bounding box by updating the doubly-connected edge list appropriately.
8. Traverse the half-edges of the doubly-connected edge list to add the cell records and the pointers to and from them.

HANDLESITEEVENT(p_i)

1. If $\mathcal{T}$ is empty, insert p_i into it (so that $\mathcal{T}$ consists of a single leaf storing p_i) and return. Otherwise, continue with steps 2– 5.
2. Search in $\mathcal{T}$ for the arc α vertically above p_i . If the leaf representing α has a pointer to a circle event in $\mathcal{Q}$, then this circle event is a false alarm and it must be deleted from $\mathcal{Q}$.
3. Replace the leaf of $\mathcal{T}$ that represents α with a subtree having three leaves. The middle leaf stores the new site p_i and the other two leaves store the site p_j that was originally stored with α . Store the tuples $\langle p_j, p_i \rangle$ and $\langle p_i, p_j \rangle$ representing the new breakpoints at the two new internal nodes. Perform rebalancing operations on $\mathcal{T}$ if necessary.
4. Create new half-edge records in the Voronoi diagram structure for the edge separating $\mathcal{V}(p_i)$ and $\mathcal{V}(p_j)$, which will be traced out by the two new breakpoints.
5. Check the triple of consecutive arcs where the new arc for p_i is the left arc to see if the breakpoints converge. If so, insert the circle event into $\mathcal{Q}$ and add pointers between the node in $\mathcal{T}$ and the node in $\mathcal{Q}$. Do the same for the triple where the new arc is the right arc.

HANDLECIRCLEEVENT(γ)

1. Delete the leaf γ that represents the disappearing arc α from $\mathcal{T}$. Update the tuples representing the breakpoints at the internal nodes. Perform rebalancing operations on $\mathcal{T}$ if necessary. Delete all circle events involving α from $\mathcal{Q}$; these can be found using the pointers from the predecessor and the successor of γ in $\mathcal{T}$. (The circle event where α is the middle arc is currently being handled, and has already been deleted from $\mathcal{Q}$.)
2. Add the center of the circle causing the event as a vertex record to the doubly-connected edge list $\mathcal{D}$ storing the Voronoi diagram under construction. Create two half-edge records corresponding to the new breakpoint of the beach line. Set the pointers between them appropriately. Attach the three new records to the half-edge records that end at the vertex.
3. Check the new triple of consecutive arcs that has the former left neighbor of α as its middle arc to see if the two breakpoints of the triple converge. If so, insert the corresponding circle event into $\mathcal{Q}$. and set pointers between the new circle event in $\mathcal{Q}$ and the corresponding leaf of $\mathcal{T}$. Do the same for the triple where the former right neighbor is the middle arc.

1 Maillage et triangulation

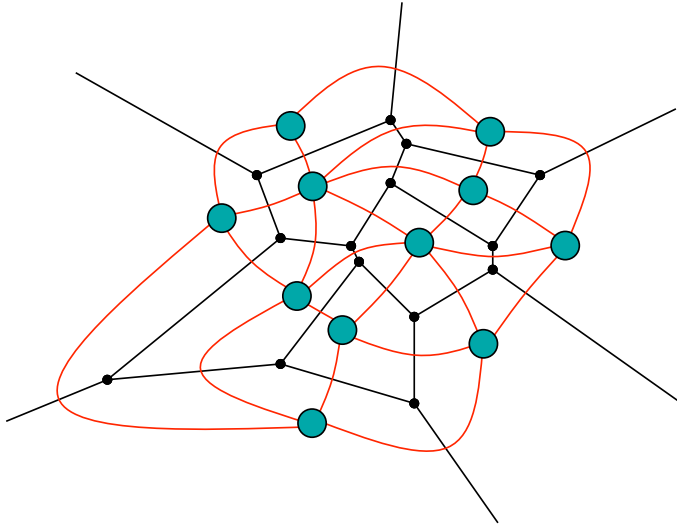
- Généralités
- Diagrammes de Voronoï
- Triangulation de Delaunay

2 Enveloppe convexe 3D

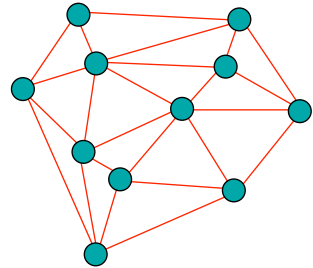
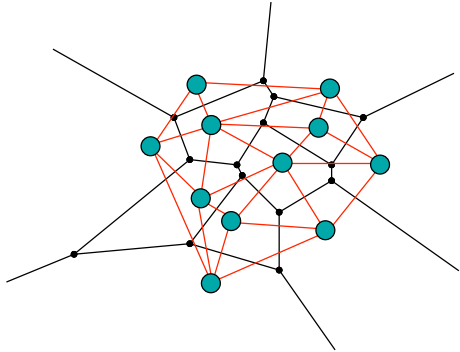
Soit P un ensemble de n points et $\mathcal{V}or(P)$, son diagramme de Voronoï. On définit $\mathcal{G}(P)$ comme le **graphe dual** de $\mathcal{V}or(P)$.

On a donc que

- Pour **chaque cellule** de $\mathcal{V}or(P)$, il y a **un sommet** dans $\mathcal{G}(P)$.
- Pour **chaque arête** de $\mathcal{V}or(P)$, il y a **une arête** dans $\mathcal{G}(P)$.
- Pour **chaque sommet** de $\mathcal{V}or(P)$, il y a **une cellule bornée** dans $\mathcal{G}(P)$.

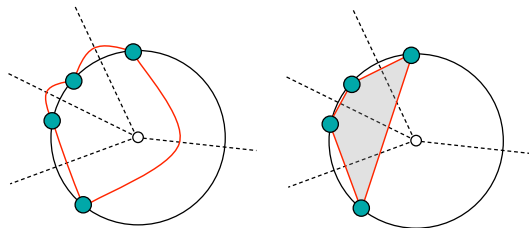


On appelle **graphe de Delaunay**, noté $\mathcal{GD}(P)$, le graphe $\mathcal{G}(P)$ où les arêtes sont **des segments de droite entre les sommets**.

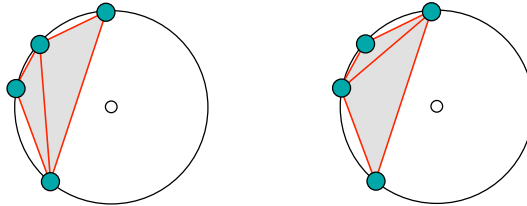


Le graphe de Delaunay n'est **pas nécessairement** une triangulation.

Si un sous-ensemble de **plus de trois points de P** se trouve sur un **même cercle vide**, ces points forment **un polygone convexe vide**. Cela se produit lorsqu'un sommet de $\mathcal{V}_{or}(P)$ a un degré supérieur à 3.



La **triangulation de Delaunay**, notée $\mathcal{TD}(P)$, est obtenue en ajoutant 0 ou plus arêtes au graphe de Delaunay $\mathcal{GD}(P)$.



On ajoute le segment **maximisant localement l'angle minimal**.

La triangulation de Delaunay est **toujours légale**. Elle est aussi **angle-optimale**.

Le problème de trouver une triangulation qui **maximise l'angle minimum** est donc réduit à **trouver une triangulation de Delaunay**.

Algorithme Trouver une triangulation de Delaunay pour $\mathcal{P}$

ENTRÉES: Ensemble de points $\mathcal{P}$

SORTIES: $\mathcal{TD}(\mathcal{P})$

- 1: Calculer $\mathcal{Vor}(\mathcal{P})$
 - 2: Trouver le graphe dual $\mathcal{G}(\mathcal{P})$
 - 3: Redresser les arêtes pour obtenir $\mathcal{GD}(\mathcal{P})$
 - 4: Ajouter des arêtes pour terminer la triangulation au besoin
-

D'autres méthodes permettent d'obtenir la triangulation de Delaunay pour un ensemble de points P .

La **méthode incrémentale aléatoire** en est une, et elle a l'avantage de ne pas nécessiter le calcul du diagramme de Voronoï.

Algorithme Trouver une triangulation de Delaunay pour $\mathcal{P}$ par la méthode incrémentale aléatoire

ENTRÉES: Ensemble de points $\mathcal{P}$

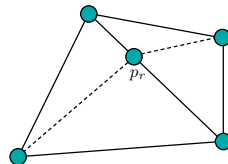
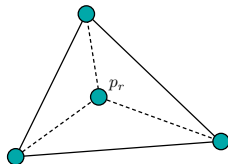
SORTIES: $\mathcal{TD}(\mathcal{P})$

- 1: Initialiser la triangulation avec un triangle assez gros, de telle sorte qu'il entoure tous les points de $\mathcal{P}$
 - 2: Choisir un point de $\mathcal{P}$ au hasard : $\mathbf{p}_r$
 - 3: Trouver le triangle dans lequel se trouve $\mathbf{p}_r$
 - 4: Subdiviser ce triangle en 3 triangles plus petits ayant $\mathbf{p}_r$ comme sommet
 - 5: Tourner des arêtes jusqu'à ce que toutes les arêtes soient légales
 - 6: Répéter les étapes 2, 3, 4 et 5 jusqu'à ce que tous les points de $\mathcal{P}$ soient dans la triangulation
 - 7: Enlever les 3 points du triangle englobant et tous les triangles reliés
-

À l'étape 4, avant l'ajout de p_r et la substitution, **toutes les arêtes sont légales**.

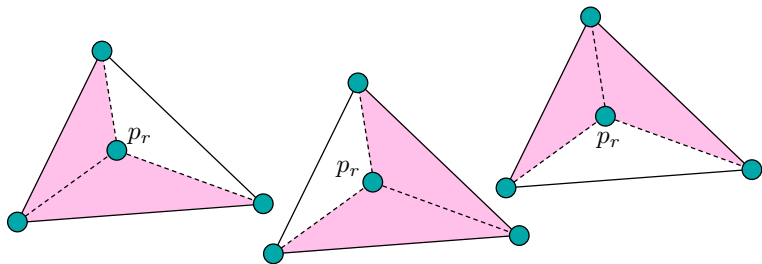
Deux cas sont alors possibles lors de l'ajout d'un point :

- p_r tombe à l'intérieur d'un triangle
- p_r tombe sur une arête existante

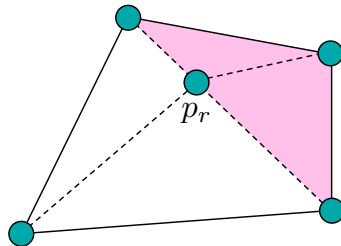
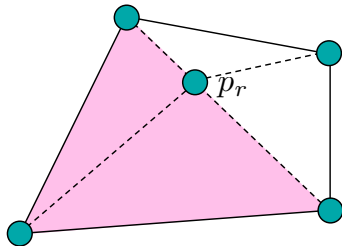


À l'étape 5, on doit déterminer **quelles arêtes sont illégales**. Or, après la subdivision, **toutes les arêtes adjacentes à p_r sont légales**.

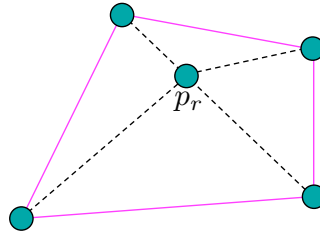
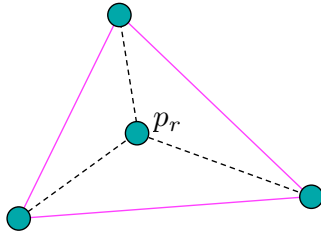
Si p_r tombe à l'intérieur d'un triangle, les **quadrilatères ne sont pas convexes**. On ne peut donc permuter aucune arête.



Si p_r tombe sur une arête existante, **deux des quadrilatères sont convexes**. Il est par contre impossible de permuter des arêtes.



Une arête peut devenir illégale si un de ses triangles adjacents a changé. Les arêtes des nouveaux triangles qui sont non adjacentes à p_r peuvent donc être illégales.



Il faut donc **vérifier chacun des nouveaux triangles** créés :

- Est-ce qu'on a un quadrilatère convexe ?
- Si oui, a-t-on une arête légale ? Si ce n'est pas le cas, on permute.

Notons que comme la permutation crée deux nouveaux triangles, il faut **reprendre la vérification pour ceux-ci**.

1 Maillage et triangulation

2 Enveloppe convexe 3D

À la section précédente, on a vu que l'enveloppe convexe d'un ensemble P contenant n points dans le plan est **un polygone convexe** dont les sommets sont des points de P . Ce faisant, l'enveloppe convexe **contenait au plus n sommets**.

En 3D, on peut faire une affirmation semblable : l'enveloppe convexe d'un ensemble P de n points 3D est **un polyèdre convexe** dont les sommets sont des points de P et qui **contient au plus n sommets**.

Dans le cas planaire, la borne supérieure sur le nombre de sommets avait comme conséquence que **la complexité géométrique de l'enveloppe convexe était linéaire**, puisque le nombre d'arêtes d'un polygone est égale au nombre de sommets de celui-ci.

Une fois passé en 3D, cette affirmation ne tient plus : le nombre d'arêtes d'un polyèdre peut être **supérieur à son nombre de sommets**.

Fort heureusement, **cette différence ne peut être trop grande**. On dispose même d'un théorème donnant une borne supérieure pour le nombre d'arêtes et le nombre de faces d'un polyèdre.

Théorème

Soit $\mathcal{P}$ un polyèdre convexe ayant n sommets. Le **nombre d'arêtes** de $\mathcal{P}$ est alors au plus $3n - 6$ et le **nombre de faces** de $\mathcal{P}$ est au plus $2n - 4$.

En combinant le théorème précédent avec le fait que l'enveloppe convexe d'un ensemble P de points 3D est un polyèdre convexe dont les sommets sont des éléments de P , on obtient que **la complexité géométrique de l'enveloppe convexe** d'un ensemble contenant n points 3D est **linéaire**.

Attention ! On parle bien ici de la complexité de la structure géométrique (nombre d'arêtes et de sommets), et non du temps d'exécution de l'algorithme de calcul de l'enveloppe convexe.

Soit P un ensemble de n points 3D. On calculera l'enveloppe convexe $\mathcal{CH}(P)$ en utilisant un **algorithme incrémental aléatoire**.

La première étape de la construction incrémentale consiste à **choisir quatre points de P qui ne soient pas coplanaires**, de telle sorte que leur enveloppe convexe soit un tétraèdre.

Pour obtenir ceux-ci, on conserve p_1 et p_2 , les deux premiers points de P , puis on parcourt l'ensemble à la recherche d'un point p_3 qui ne soit **pas colinéaire avec p_1 et p_2** .

On continue notre chemin pour ensuite trouver un point p_4 qui ne soit **pas coplanaire avec p_1 , p_2 et p_3** .

S'il est impossible de trouver quatre points non coplanaires, alors **les données de P reposent sur un même plan** et on peut simplement utiliser la technique de la section précédente pour évaluer l'enveloppe convexe.

On évalue ensuite une **permutation aléatoire des points restants** : $p_5, \dots, p_n$. L'algorithme s'attardera aux points un par un dans cet ordre aléatoire en maintenant à jour l'enveloppe convexe à chaque étape.

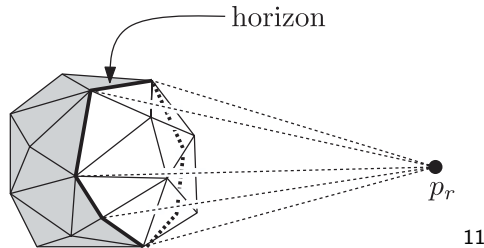
Pour un entier $r \geq 1$, on définit l'ensemble $P_r = \{p_1, \dots, p_r\}$. À une étape donnée de l'algorithme, **on aura à ajouter le point p_r à l'enveloppe convexe de P_{r-1} .**

On devra donc transformer $\mathcal{CH}(P_{r-1})$ en $\mathcal{CH}(P_r)$.

On aura alors **deux situations possibles** :

- Si p_r se trouve à l'intérieur ou sur la frontière de $\mathcal{CH}(P_{r-1})$, alors on a $\mathcal{CH}(P_{r-1}) = \mathcal{CH}(P_r)$ et aucun calcul n'est nécessaire.
- Si p_r se trouve à l'extérieur de $\mathcal{CH}(P_{r-1})$, alors... la vie ne sera pas si simple !

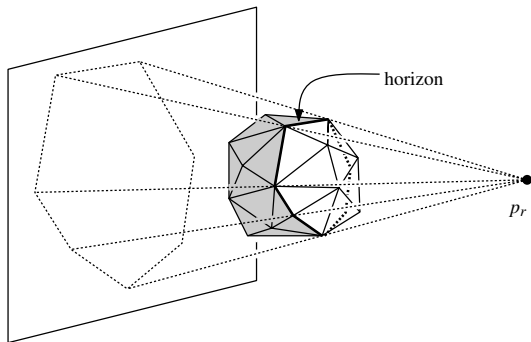
Imaginons que l'on se trouve à la position p_r et que l'on regarde $\mathcal{CH}(P_{r-1})$. **On pourra alors voir certaines faces de l'enveloppe convexe**, tandis que d'autres seront invisibles.



11

Les faces visibles forment la **région visible** de p_r sur $\mathcal{CH}(P_{r-1})$. Cette région est bornée par un ensemble fermé d'arêtes de $\mathcal{CH}(P_{r-1})$. On appelle **horizon** de p_r ce groupe d'arêtes.

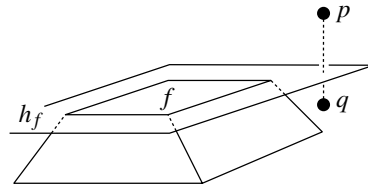
La **projection de l'horizon** donne la frontière du polygone convexe obtenu en projetant $\mathcal{CH}(P_{r-1})$ sur un plan en utilisant p_r comme centre de projection.



Qu'entend-on géométriquement lorsqu'on dit qu'une face de $\mathcal{CH}(P_{r-1})$ est **visible à partir d'un point** p_r ? Pour comprendre, considérons h_f , le plan de support de la face f de $\mathcal{CH}(P_{r-1})$.

En raison de sa convexité, on a que $\mathcal{CH}(P_{r-1})$ **se trouve entièrement dans un des deux demi-espaces** définis par h_f .

On aura alors que la face f sera visible à partir d'un point si celui-ci se trouve dans le demi-espace ne contenant pas $\mathcal{CH}(P_{r-1})$.



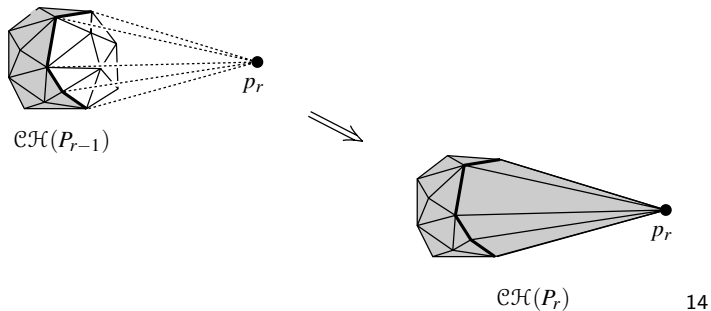
f is visible from p ,
but not from q

13

L'horizon de p_r joue un rôle crucial quand vient le temps de **transformer** $\mathcal{CH}(P_{r-1})$ en $\mathcal{CH}(P_r)$, puisqu'il sépare les faces de l'enveloppe convexe en deux catégories :

- Celles qui seront conservées (**les faces invisibles**) ; et
- Celles qui seront remplacées (**les faces visibles**).

Ces dernières seront remplacées par des faces reliant p_r à son horizon.



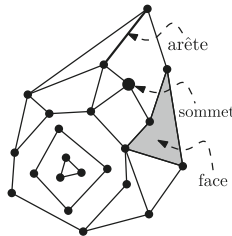
Avant de poursuivre avec le processus de mise à jour de $\mathcal{CH}(P_{r-1})$ suite à l'ajout du point p_r , on doit **décider d'une structure de données permettant de stocker l'enveloppe convexe**.

Contrairement au cas 2D, représenter l'enveloppe convexe comme une simple liste d'arêtes n'est pas une bonne idée.

On utilisera plutôt la **liste d'arêtes doublement chaînée (DCEL)**, qui, à la base, est une structure élaborée pour **stocker une subdivision du plan**.

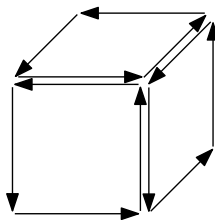
Ainsi, une DCEL sert à représenter en mémoire **une subdivision planeire**, mais son fonctionnement se prête très bien au stockage d'une enveloppe convexe 3D.

Une DCEL contient un enregistrement pour chaque **face**, chaque **arête** et chaque **sommet** de l'enveloppe convexe.



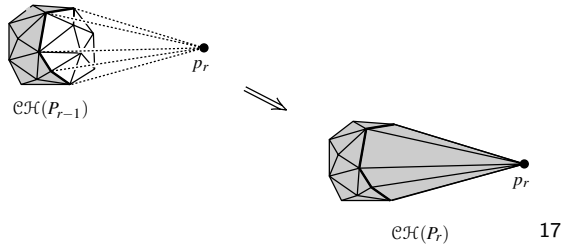
La frontière d'un polyèdre convexe peut être **interprétée comme une subdivision plane**. Ce faisant, la DCEL est une structure appropriée pour stocker une enveloppe convexe 3D en mémoire.

On construira la liste de manière à ce que les arêtes soient orientées dans le sens **antihoraire** lorsqu'on les observe depuis l'extérieur du polyèdre.



16

Revenons donc à nos moutons... on souhaite ajouter un point p_r à l'enveloppe convexe $\mathcal{CH}(P_{r-1})$.



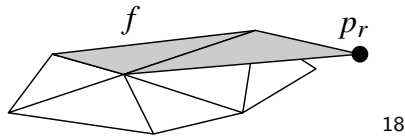
On dispose donc d'une **DCEL** représentant $\mathcal{CH}(P_{r-1})$ et on souhaite la modifier pour obtenir la DCEL de $\mathcal{CH}(P_r)$.

Si l'on connaissait **toutes les faces de $\mathcal{CH}(P_{r-1})$ qui sont visibles** depuis le point p_r , il serait facile de

1. **Retirer** ces faces de la DCEL ;
2. **Évaluer** les faces reliant p_r à l'horizon ;
3. **Ajouter** ces nouvelles faces à la DCEL

L'ensemble de ces opérations peut se faire en temps $O(n)$.

Il resterait seulement à s'assurer que l'ajout des nouvelles faces **n'a pas créé de surface coplanaire** dans $\mathcal{CH}(P_r)$.



Cette situation survient si p_r se trouve sur le plan de support d'une face f dans $\mathcal{CH}(P_{r-1})$. Or, selon notre définition de la visibilité, une telle face n'est pas visible depuis la position p_r .

La conséquence sera que, comme f n'est pas visible, elle restera inchangée dans la DCEL, mais **des arêtes reliant p_r aux sommets de f faisant partie de l'horizon** seront ajoutés.

Comme cette nouvelle face est coplanaire avec f , on doit les combiner afin de **former une seule face**.

Pour arriver ici, on a fait la supposition que l'**on connaissait toutes les faces de $\mathcal{CH}(P_{r-1})$ qui sont visibles** depuis le point p_r , mais comment obtient-on celles-ci ?

Bien entendu, on pourrait, pour chaque point ajouté, **tester l'ensemble des faces** pour savoir de quel côté se point se situe. Cela mènerait toutefois à un temps d'exécution $O(n^2)$... tâchons de faire mieux.

L'idée sera ici de **prendre les devants sur le problème** : en plus d'une DCEL représentant l'enveloppe convexe, on conservera en mémoire de l'information supplémentaire :

- Pour chaque face $f \in \mathcal{CH}(P_{r-1})$, on stocke un ensemble

$$P_{\text{conflit}}(f) \subseteq \{p_{r+1}, p_{r+2}, \dots, p_n\}$$

contenant **tous les points qui peuvent voir f** .

- Inversement, **pour chaque point p_t avec $t > r$** , on stocke un ensemble

$$F_{\text{conflit}}(p_t) \subseteq \mathcal{CH}(P_r)$$

contenant **toutes les faces visibles depuis p_t** .

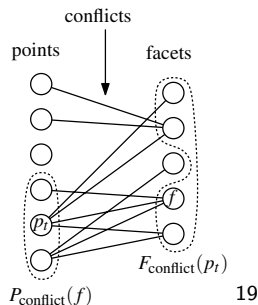
On dira qu'un point $p \in P_{\text{conflit}}(f)$ est **en conflit avec la face f** puisque p et f ne peuvent pas être ensemble dans l'enveloppe convexe. Lorsqu'on ajoute un point $p \in P_{\text{conflit}}(f)$ à l'enveloppe convexe, on doit en supprimer la face f .

On dira que $P_{\text{conflit}}(f)$ et $F_{\text{conflit}}(p_t)$ sont des **listes de conflits**.

Les listes de conflits sont maintenues à jour à l'aide d'un **graphe de conflits** noté $\mathcal{G}$.

Celui-ci possède **un sommet pour chaque point** de P n'ayant pas été inséré dans l'enveloppe convexe de même qu'**un sommet pour chaque face** de l'enveloppe convexe courante.

$\mathcal{G}$ possède aussi **une arête pour chaque conflit** entre un point et une face.



19

En d'autres mots, il y aura une arête entre un point $p_t \in P$ et une face $f \in \mathcal{CH}(P_r)$ si $r < t$ et f est visible depuis p_t .

À l'aide du graphe $\mathcal{G}$, on peut retrouver l'ensemble $F_{\text{conflict}}(p_t)$ pour un point p_t quelconque et l'ensemble $P_{\text{conflict}}(f)$ pour une face f quelconque, et ce en temps $O(n)$.

Ainsi, lorsqu'on ajoute p_r à $\mathcal{CH}(P_{r-1})$, il suffit de récupérer $F_{\text{conflict}}(p_r)$ à partir de $\mathcal{G}$ et de remplacer les anciennes faces par les nouvelles qui connectent p_r à l'horizon.

L'initialisation de $\mathcal{G}$ pour $\mathcal{CH}(P_4)$ peut être faite en temps $O(n)$. Il suffit de parcourir P et de déterminer quelles faces de $\mathcal{CH}(P_4)$ les points peuvent voir.

La mise à jour du graphe $\mathcal{G}$ suite à l'ajout du point p_r se fait en trois étapes :

1. On élimine du graphe les sommets et les arêtes incidents associés aux faces de $\mathcal{CH}(P_{r-1})$ qui disparaissent de l'enveloppe convexe.
2. On élimine du graphe le sommet associé au point p_r .
3. On ajoute des sommets au graphe pour chaque nouvelle face qui relie p_r à l'horizon.

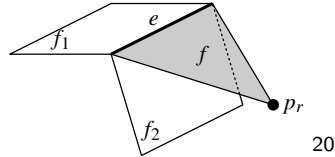
Il reste alors à trouver les listes de conflits associées à ces nouvelles faces.

Les faces créées suite à l'insertion de p_r **sont toutes des triangles**, à l'exception de celles reposant sur le plan de support d'une face de $\mathcal{CH}(P_{r-1})$ et qui ont été combinées avec celles-ci.

La liste de conflits associée à une telle face **demeure la même que celle de la face de $\mathcal{CH}(P_{r-1})$** avec laquelle elle a été combinée.

Trouver la liste de conflits associée à une nouvelle face triangulaire sera **moins trivial...**

Soit f une nouvelle face incidente à p_r ajoutée dans $\mathcal{CH}(P_r)$. Si f est visible depuis un point p_t , alors p_t voit nécessairement l'arête $\vec{e}$ de f qui est opposée à p_r .



Cette arête $\vec{e}$ faisait partie de l'horizon du point p_r et était ainsi nécessairement présente dans $\mathcal{CH}(P_{r-1})$.

Or, comme $\mathcal{CH}(P_{r-1}) \subseteq \mathcal{CH}(P_r)$, l'arête $\vec{e}$ devait aussi être visible depuis p_t dans $\mathcal{CH}(P_{r-1})$. Pour que cela se produise, **au moins une des deux faces incidentes à $\vec{e}$ dans $\mathcal{CH}(P_{r-1})$ devait être visible depuis p_t .**

Cela implique que l'on peut trouver la liste de conflits associée à la face f en testant **seulement les points en conflit avec les deux faces incidentes à l'arête $\vec{e}$ dans $\mathcal{CH}(P_{r-1})$.**

Algorithme 3: ENVELOPPECONVEXE3D

Entrées : Un ensemble $P = \{p_1, p_2, \dots, p_n\}$ où $p_i \in \mathbb{R}^3$
Sorties : L'enveloppe convexe $\mathcal{CH}(P)$ de P

- 1 Trouver quatre points p_1, p_2, p_3, p_4 dans P qui ne soient pas coplanaires;
- 2 $\mathcal{C} \leftarrow \mathcal{CH}(p_1, p_2, p_3, p_4)$;
- 3 Obtenir une permutation aléatoire des points $p_5, p_6, \dots, p_n$;
- 4 Initialiser le graphe de conflits $\mathcal{G}$ avec toutes les paires (p_t, f) visibles, où f est une face de $\mathcal{C}$ et $t > 4$;
- 5 **pour** $r = 5$ à n **faire**
- 6 **si** $F_{\text{conflit}}(p_r)$ n'est pas vide (c-à-d si $p_r \notin \mathcal{C}$) **alors**
- 7 Ajouter p_r à $\mathcal{C}$;
- 8 Retirer les faces dans $F_{\text{conflit}}(p_r)$ de $\mathcal{C}$;
- 9 Créer une liste ordonnée $\mathcal{L}$ des arêtes formant l'horizon de p_r ;
- 10 **pour chaque** arête $e \in \mathcal{L}$ **faire**
- 11 Relier e à p_r en créant une face triangulaire f et l'ajouter à $\mathcal{C}$;
- 12 **si** f est coplanaire avec la face voisine f' le long de e **alors**
- 13 Combiner f et f' en une seule face dont la liste de conflits est la même que pour f' ;
- 14 **sinon**
- 15 Créer un sommet pour f dans $\mathcal{G}$;
- 16 $P(e) \leftarrow P_{\text{conflit}}(f_1) \cup P_{\text{conflit}}(f_2)$ où f_1 et f_2 sont les deux faces incidente à e dans $\mathcal{CH}(P_{r-1})$;
- 17 **pour chaque** point $p \in P(e)$ **faire**
- 18 Si f est visible depuis p , ajouter la paire (p, f) dans $\mathcal{G}$;
- 19 Supprimer le sommet associé à p_r de $\mathcal{G}$;
- 20 Supprimer les sommets correspondants aux faces dans $F_{\text{conflit}}(p_r)$ de $\mathcal{G}$ ainsi que leurs arêtes incidentes;
- 21 **retourner** $\mathcal{C}$;

Finalement, on s'attarde à **la complexité de l'algorithme** précédent. On acceptera ces résultats sans démonstration.

Soit P un ensemble de n points dans $\mathbb{R}^3$. Le **nombre total de faces générées par l'algorithme** est au plus $6n - 20$ et le calcul de l'enveloppe convexe associée à P s'effectue **en un temps attendu de $O(n \log n)$** , selon le résultat de la permutation aléatoire.

- 1 Maillage et triangulation
- 2 Enveloppe convexe 3D